

Praktikumsaufgaben 8

Kontrollflussgraph-basiertes Testen · White-Box-Methoden

◆ Grundbegriffe: Kontrollflussgraph-basiertes Testen

Ein Kontrollflussgraph (KFG) stellt den Programmablauf als gerichteten Graphen dar.

Knoten = Anweisungen / Blöcke | Kanten = mögliche Übergänge (Verzweigungen)

Wichtige Überdeckungskriterien:

- Anweisungsüberdeckung (C0): Jede Anweisung wird mind. einmal ausgeführt.
- Zweigüberdeckung (C1): Jede Kante (TRUE- und FALSE-Zweig jeder Bedingung) wird mind. einmal durchlaufen.
- Einfache Bedingungsüberdeckung (C1b): Jede atomare Teilbedingung nimmt mind. einmal TRUE und einmal FALSE an.
- Mehrfache Bedingungsüberdeckung (C2): Alle Kombinationen aller Teilbedingungen werden getestet.

A1

Lohnsteuerberechnung – Kontrollflussgraphbasiertes Testen

Aufgabenstellung

Für Berechnung der Lohnsteuer werden die folgenden Regeln verwendet:

- Für den Bruttolohn unter 1000 wird keine Lohnsteuer erhoben.
- Für den Bruttolohn zwischen 1000 und 2500 Euro wird die Lohnsteuer in Höhe von 12% erhoben.
- Für den Bruttolohn über 2500 Euro wird die Lohnsteuer in Höhe von 15% erhoben.

1. Erstellen Sie den Kontrollflussgraph für dieses Programm.
2. Welche Testparameterkombinationen für 100% Anweisungs- und 100% Zweigüberdeckung?
3. Welche Testparameterkombinationen für 100% einfache Bedingungsüberdeckung?

◆ Pseudocode

```
FUNCTION BerechneLohnsteuer(Nettolohn)
  IF (Nettolohn < 1000)                                // Knoten 1
  THEN
    Steuer = 0;                                         // Knoten 2
  ENDIF
  IF (Nettolohn >= 1000 UND Nettolohn <= 2500)         // Knoten 3
  THEN
    Steuer = Nettolohn * 0,12;                         // Knoten 4
  ENDIF
  IF (Nettolohn > 2500)                                // Knoten 5
```

```

THEN
    Steuer = Nettolohn * 0,15;           // Knoten 6
ENDIF
RETURN Steuer;                         // Knoten 7
END FUNCTION

```

◆ 1. Kontrollflussgraph

Knoten und Kanten des Kontrollflussgraphen:

```

K1: IF Nettolohn < 1000 —TRUE—▶ K2: Steuer = 0
K1      —FALSE—▶ K3
K2      —————▶ K3
K3: IF Nettolohn >= 1000 AND <= 2500 —TRUE—▶ K4: Steuer = Nettolohn * 0,12
K3      —FALSE—▶ K5
K4      —————▶ K5
K5: IF Nettolohn > 2500 —TRUE—▶ K6: Steuer = Nettolohn * 0,15
K5      —FALSE—▶ K7
K6      —————▶ K7: RETURN Steuer

```

Kanten gesamt: 9 | Knoten gesamt: 7

Zyklomatische Komplexität: $V(G) = \text{Kanten} - \text{Knoten} + 2 = 9 - 7 + 2 = 4$

◆ 2. Anweisungsüberdeckung (C0) & Zweigüberdeckung (C1)

Für C0 reicht ein Testfall pro erreichbarem Pfad. Da alle 3 IF-Zweige unabhängig sequenziell ausgewertet werden, braucht man genau 3 Testfälle für vollständige Zweigüberdeckung:

Testfall	Nettolohn	TRUE-Zweige	Ausgeführte Knoten	Steuer (Soll)	Überdeckung
T1	500 €	K1=TRUE	K1→K2→K3→K5→K7	0 €	C0+C1: K1-TRUE
T2	1500 €	K3=TRUE	K1→K3→K4→K5→K7	180 €	C0+C1: K1-FALSE, K3-TRUE
T3	3000 €	K5=TRUE	K1→K3→K5→K6→K7	450 €	C0+C1: K3-FALSE, K5-TRUE

✓ Ergebnis C0 + C1

3 Testfälle decken alle Anweisungen UND alle Zweige (TRUE + FALSE) ab:

T1 (500€) → K1-TRUE-Zweig, K3-FALSE-Zweig, K5-FALSE-Zweig

T2 (1500€) → K1-FALSE-Zweig, K3-TRUE-Zweig, K5-FALSE-Zweig

T3 (3000€) → K1-FALSE-Zweig, K3-FALSE-Zweig, K5-TRUE-Zweig

Alle 7 Knoten und 9 Kanten werden damit abgedeckt → 100% C0 und 100% C1.

◆ 3. Einfache Bedingungsüberdeckung (C1b)

Die drei Bedingungen sind atomar (jeweils nur eine Teilbedingung). Für C1b muss jede Bedingung mind. einmal TRUE und mind. einmal FALSE sein — das wird bereits durch die obigen 3 Testfälle T1–T3 erreicht:

Bedingung	T1 (500€)	T2 (1500€)	T3 (3000€)
Nettolohn < 1000	TRUE ✓	FALSE ✓	FALSE
Nettolohn >= 1000 AND <= 2500	FALSE ✓	TRUE ✓	FALSE
Nettolohn > 2500	FALSE ✓	FALSE	TRUE ✓

✓ Ergebnis C1b

Die gleichen 3 Testfälle T1, T2, T3 erreichen auch 100% einfache Bedingungsüberdeckung, da jede der 3 atomaren Bedingungen sowohl TRUE als auch FALSE annimmt.

Aufgabenstellung

Die Funktion berechnet Rabatt basierend auf: Warenkorbwert, Rabattcode und Gültigkeitsdatum.

IF (warenkorbwert >= 50 UND rabattcode == "SOMMER20" UND Heute <= gueltig_bis) THEN rabatt = 0.1
ELSE rabatt = 0

1. Erstellen Sie den Kontrollflussgraph für diese Funktion.
2. Testparameterkombinationen für 100% Anweisungs- und 100% Zweigüberdeckung?
3. Testparameterkombinationen für 100% einfache Bedingungsüberdeckung?
4. Testparameterkombinationen für 100% mehrfache Bedingungsüberdeckung?

◆ **Pseudocode**

```
FUNCTION berechneRabatt(warenkorbwert, rabattcode, gueltig_bis)
  heute = aktuelles Datum
  IF (warenkorbwert >= 50                                // Bedingung B1
      UND rabattcode == "SOMMER20"                       // Bedingung B2
      UND Heute <= gueltig_bis)                          // Bedingung B3
  THEN
    rabatt = 0.1    // 10% Rabatt                        // Knoten 2
  ELSE
    rabatt = 0      // Kein Rabatt                        // Knoten 3
  RETURN rabatt    // Knoten 4
END FUNCTION
```

◆ **1. Kontrollflussgraph**

Knoten und Kanten:

```

K1: IF (B1 AND B2 AND B3) —TRUE—▶ K2: rabatt = 0.1
K1      —FALSE—▶ K3: rabatt = 0
K2      —————▶ K4: RETURN rabatt
K3      —————▶ K4
```

Knoten: 4 | Kanten: 4 | $V(G) = 4 - 4 + 2 = 2$

Die Gesamtbedingung ist eine zusammengesetzte Bedingung mit 3 Teilbedingungen (B1, B2, B3), alle verknüpft mit UND (Short-Circuit-Evaluation beachten).

◆ **2. Anweisungsüberdeckung (C0) & Zweigüberdeckung (C1)**

Es gibt nur 2 Zweige (THEN und ELSE). Zwei Testfälle genügen:

Testfall	Warenkorbwert	Rabattcode	Gültig bis	Ergebnis	Überdeckung
T1	80 €	SOMMER20	Zukunft	rabatt = 0.1 (10%)	THEN-Zweig (K1-TRUE, K2, K4)
T2	30 €	SOMMER20	Zukunft	rabatt = 0 (0%)	ELSE-Zweig (K1-FALSE, K3, K4)

✓ Ergebnis C0 + C1

T1: Alle Bedingungen TRUE → THEN-Zweig → 10% Rabatt
T2: B1 FALSE (30 < 50) → Short-Circuit → ELSE-Zweig → 0% Rabatt
Damit sind alle 4 Knoten und alle 4 Kanten abgedeckt.

◆ 3. Einfache Bedingungsüberdeckung (C1b)

Jede der 3 Teilbedingungen B1, B2, B3 muss mind. einmal TRUE und mind. einmal FALSE sein:

Testfall	Warenkorbwert	Rabattcode	Gültig bis	B1	B2	B3	Rabatt
T1	80 €	SOMMER20	Zukunft	TRUE	TRUE	TRUE	0.1 (10%)
T2	30 €	SOMMER20	Zukunft	FALSE	TRUE	–	0 (0%)
T3	80 €	WINTER10	Zukunft	TRUE	FALSE	–	0 (0%)
T4	80 €	SOMMER20	Vergangenheit	TRUE	TRUE	FALSE	0 (0%)

✓ Ergebnis C1b

T1: B1=T, B2=T, B3=T → alle drei Teilbedingungen einmal TRUE
T2: B1=FALSE → B1 einmal FALSE (B2, B3 durch Short-Circuit nicht ausgewertet)
T3: B2=FALSE → B2 einmal FALSE
T4: B3=FALSE → B3 einmal FALSE
→ 4 Testfälle für vollständige einfache Bedingungsüberdeckung.

◆ 4. Mehrfache Bedingungsüberdeckung (C2)

Alle $2^3 = 8$ Kombinationen der 3 Teilbedingungen müssen getestet werden:

TF	B1 (≥50€)	B2 (SOMMER20)	B3 (Datum ok)	Gesamt	Rabatt	Warenkorbwert	Rabattcode	Datum
T1	TRUE	TRUE	TRUE	TRUE	10%	80€	SOMMER20	Zukunft
T2	TRUE	TRUE	FALSE	FALSE	0%	80€	SOMMER20	Vergang.
T3	TRUE	FALSE	TRUE	FALSE	0%	80€	WINTER10	Zukunft
T4	TRUE	FALSE	FALSE	FALSE	0%	80€	WINTER10	Vergang.

TF	B1 (≥50€)	B2 (SOMMER20)	B3 (Datum ok)	Gesamt	Rabatt	Warenkorbwert	Rabattcode	Datum
T5	FALSE	TRUE	TRUE	FALSE	0%	30€	SOMMER20	Zukunft
T6	FALSE	TRUE	FALSE	FALSE	0%	30€	SOMMER20	Vergang.
T7	FALSE	FALSE	TRUE	FALSE	0%	30€	WINTER10	Zukunft
T8	FALSE	FALSE	FALSE	FALSE	0%	30€	WINTER10	Vergang.

✓ Ergebnis C2

8 Testfälle decken alle $2^3 = 8$ Kombinationen ab → 100% mehrfache Bedingungsüberdeckung.

Hinweis: Bei Short-Circuit-Auswertung werden T5–T8 (B1=FALSE) B2 und B3 ggf. nicht ausgewertet.

Für echte C2-Abdeckung muss die Auswertungsreihenfolge und Short-Circuit-Verhalten bekannt sein.

Aufgabenstellung

Unten ist der Pseudocode für den Algorithmus "Bubble-Sort" aufgeführt.

1. Erstellen Sie den Kontrollflussgraph für dieses Programm.
2. Welche Testparameterkombinationen für 100% Anweisungs- und 100% Zweigüberdeckung?

◆ **Pseudocode**

```

FUNCTION BubbleSort(arr)
    n = LENGTH(arr)                // K1: Initialisierung
    FOR i FROM 1 TO n-1 DO          // K2: Äußere Schleife (Bedingung i <= n-1)
        FOR j FROM 1 TO n-i DO      // K3: Innere Schleife (Bedingung j <= n-i)
            IF arr[j] > arr[j+1] THEN // K4: Vergleich
                SWAP(arr[j], arr[j+1]) // K5: Tausch
            END IF                  // K6: Nach IF (kein Tausch)
        END FOR                    // zurück zu K3
    END FOR                        // zurück zu K2
END FUNCTION                      // K7: Ende

```

◆ **1. Kontrollflussgraph**

Knoten und Kanten:

```

K1: n = LENGTH(arr) —————▶ K2
K2: i <= n-1? —TRUE—▶ K3 | —FALSE—▶ K7 (Ende)
K3: j <= n-i? —TRUE—▶ K4 | —FALSE—▶ K2 (nächste i-Iteration)
K4: arr[j] > arr[j+1]? —TRUE—▶ K5: SWAP | —FALSE—▶ K6
K5: SWAP —————▶ K6
K6: j++ —————▶ K3 (zurück)
K7: RETURN (Ende)

```

Knoten: 7 | Kanten: 9 | $V(G) = 9 - 7 + 2 = 4$

4 unabhängige Pfade (entspricht zyklomatischer Komplexität).

◆ **2. Anweisungsüberdeckung (C0) & Zweigüberdeckung (C1)**

Für C0 reicht ein einziger Testfall mit einem Array, das mindestens einen Tausch erfordert. Für C1 brauchen wir zusätzlich den Fall, dass kein Tausch stattfindet:

Testfall	Eingabe array	Beschreibung	Ausgeführte Knoten / Zweige	Erwartete Ausgabe
T1	[3, 1, 2]	Array mit Tauschbedarf	K1→K2(T)→K3(T)→K4(T)→K5→K6→K3→K4(F)→K6→K3(F)→K2→... alle Knoten inkl. K5	[1, 2, 3]
T2	[1, 2, 3]	Bereits sortiertes Array	K1→K2(T)→K3(T)→K4(F)→K6→... K4-FALSE-Zweig, kein SWAP	[1, 2, 3]
T3	[] oder [5]	Leeres/einelementiges Array → Schleife nie ausgeführt	K1→K2(F)→K7: K2-FALSE-Zweig	[] oder [5]

✓ Ergebnis C0 + C1

T1 ([3,1,2]): Deckt alle Knoten ab inkl. SWAP (K5) → C0 erfüllt.

Außerdem: K2-TRUE, K3-TRUE, K4-TRUE und K4-FALSE abgedeckt.

T2 ([1,2,3]): K4-FALSE-Zweig (kein Tausch) explizit getestet.

T3 ([] oder [5]): K2-FALSE-Zweig (Schleife wird nie betreten) abgedeckt.

→ 3 Testfälle für 100% Anweisungsüberdeckung (C0) und 100% Zweigüberdeckung (C1).

Alle 7 Knoten und alle 9 Kanten werden dabei mindestens einmal durchlaufen.

◆ Zusammenfassung der Überdeckungskriterien

Kriterium	Symbol	Anforderung	Testfälle A1	Testfälle A2	Testfälle A3
Anweisungsüberdeckung	C0	Jede Anweisung mind. 1× ausgeführt	3 (T1–T3)	2 (T1–T2)	3 (T1–T3)
Zweigüberdeckung	C1	Jeder TRUE- und FALSE-Zweig mind. 1×	3 (T1–T3)	2 (T1–T2)	3 (T1–T3)
Einfache Bed.-überdeckung	C1b	Jede Teilbedingung mind. 1× TRUE + FALSE	3 (T1–T3)	4 (T1–T4)	–
Mehrfache Bed.-überdeckung	C2	Alle Kombinationen aller Teilbedingungen	–	8 (T1–T8)	–